Unité 7 — Cahier de laboratoire (étudiant)

Livrable C — Injection & SQLi · UE « Software Security »

Code UE 7532 14 U32 D1 · Document étudiant · 11 octobre 2026

Cadre. Application volontairement vulnérable, à exécuter uniquement en local / environnement isolé (livrable E).

Lab 7.1 — Contourner l'authentification par SQLi EXERCICE GUIDÉ

1. Objectifs

- Exploiter une injection SQL pour se connecter sans mot de passe.
- Observer la requête réellement exécutée.

2. Prérequis

- Cours U7.

3. Environnement

Image de référence (E) + Flask. Application vulnérable `app.py` fournie (et `app_fixed.py`).

4. Fichiers

APP.PY — ROUTE /LOGIN (EXTRAIT)

```
@app.route("/login")
def login():
    u = request.args.get("user","")
    p = request.args.get("pass","")
    q = "SELECT * FROM users WHERE user='%s' AND pass='%s'" % (u, p)  # VULN
    rows = db().execute(q).fetchall()
    return "OK" if rows else "echec"
```

5. Commandes

SHELL

```
# demarrer l'application vulnerable :
FLASK_APP=app.py flask run --port 5001
# dans un autre terminal :
curl -D - "http://127.0.0.1:5001/login?user=admin'--&pass=x"
curl "http://127.0.0.1:5001/login?user=x' OR '1'='1&pass=x' OR '1'='1"
```

6. Manipulations

a. Connectez-vous normalement (bon identifiant) pour voir le comportement attendu.

b. Injectez `admin'--` dans `user` : observez l'en-tête `X-Query` et la connexion réussie.

c. Essayez `' OR '1'='1` : récupérez toutes les lignes.

7. Résultats attendus

La requête devient `...user='admin'--' AND pass='x'` ; connexion en admin. Avec `OR '1'='1`, toutes les lignes remontent.

8. Explications

Le `--` commente la vérification du mot de passe ; la donnée est devenue de la syntaxe SQL faute de séparation.

9. Questions

1. Quelle est la requête exacte injectée (en-tête X-Query) ?
2. Pourquoi `--` suffit-il ?
3. Comment extraire d'autres tables (piste UNION) ?

11. Vulnérabilité & remédiation

CWE-89 (SQL injection). Remédiation : requêtes paramétrées (Lab 7.2).

12. Durée

1 h.

10. Corrigé : livrable D, Lab 7.1.

Lab 7.2 — Corriger et re-tester EXERCICE GUIDÉ

1. Objectifs

- Appliquer des requêtes paramétrées et vérifier que l'injection échoue.

2. Prérequis

- Lab 7.1.

3. Environnement

Image de référence (E) + Flask. Application vulnérable `app.py` fournie (et `app_fixed.py`).

4. Fichiers

APP_FIXED.PY — ROUTE /LOGIN (CORRECTIF)

```
rows = db().execute(
    "SELECT * FROM users WHERE user=? AND pass=?", (u, p)   # requete parametree
).fetchall()
```

5. Commandes

SHELL

```
FLASK_APP=app_fixed.py flask run --port 5003
curl "http://127.0.0.1:5003/login?user=admin'--&pass=x"      # doit echouer
curl "http://127.0.0.1:5003/login?user=admin&pass=S3cret!"    # doit reussir
```

6. Manipulations

a. Lancez la version corrigée.

b. Rejouez l'injection du Lab 7.1 : elle doit échouer.

c. Vérifiez qu'une connexion légitime fonctionne toujours.

7. Résultats attendus

L'injection renvoie « echec » ; la connexion légitime renvoie « OK ».

8. Explications

Avec le paramétrage, `admin'--` est cherché *littéralement* comme nom d'utilisateur : aucune ligne, pas d'effet de syntaxe.

9. Questions

1. Pourquoi l'injection ne fonctionne-t-elle plus ?
2. En quoi est-ce plus robuste que d'échapper les apostrophes ?
3. Quelle défense en profondeur ajouter ?

11. Vulnérabilité & remédiation

Correctif validé : requête paramétrée ; + moindre privilège DB, validation d'entrée.

12. Durée

45 min.

10. Corrigé : livrable D, Lab 7.2.

Exemples SQLi issus de meltypot ; application de labo auteur ; sorties vérifiées.

Unité 7 — Cahier de laboratoire (étudiant)

Livrable C — Injection & SQLi · UE « Software Security »

Lab 7.3 — Transfert sur OWASP Juice Shop

EXERCICE AUTONOME

1. Objectifs

Réinvestir les principes de l’injection SQL étudiés dans les Labs 7.1 et 7.2.

Analyser un formulaire de connexion sans disposer du code source serveur.

2. Prérequis

Labs 7.1 et 7.2 réalisés ; environnement Docker U07 opérationnel.

Navigateur et, facultativement, outils de développement ou Burp Suite.

3. Environnement

OWASP Juice Shop, service « juice-shop » du compose.yaml U07.

Accès : http://127.0.0.1:3000 (si le port 3000 est publié sur l’hôte).

Laboratoire volontairement vulnérable : tests uniquement sur cette instance locale.

4. Manipulations

a. Ouvrez Juice Shop et identifiez le formulaire de connexion.

b. Testez une connexion ordinaire et observez le comportement de l’application.

c. À partir du Lab 7.1, formulez une hypothèse de contournement par SQLi.

d. Essayez votre hypothèse ; adaptez-la si la syntaxe Flask ne fonctionne pas.

e. Relevez les indices observables (réponse, erreur, changement de session).

f. Comparez ce comportement à celui de app.py et app_fixed.py.

5. Questions

1. Quels champs et quelles réponses vous ont aidé à raisonner ?
2. Le payload Flask fonctionne-t-il sans modification ? Expliquez.
3. Quelles observations indiquent un succès ou un échec ?
4. Quelle mesure du Lab 7.2 préviendrait cette classe de vulnérabilité ?

6. Résultats attendus

Une démarche documentée : hypothèse, essais, observations et conclusion.

La réussite d’un contournement est un objectif, pas un prérequis pour expliquer.

Aucun payload imposé : l’argumentation compte autant que le résultat.

7. Durée et remise

30 à 45 min. Remettre les requêtes testées et un court compte rendu.

Ne pas viser d’autres sites ni des instances non autorisées.